



INSTITUTO JEAN PIAGET DO SUL – CAMPO DE ALMADA
ESCOLA SUPERIOR DE TECNOLOGIAS E GESTÃO

CURSO: ENGENHARIA INFORMÁTICA

UNIDADE CURRICULAR
ENGENHARIA DE SOFTWARE
SEMESTRE 2, ANO LECTIVO 2025/2026

SkoolBay

Marketplace de Serviços entre Estudantes Universitários

Repositório: github.com/pereira2001/ESTG-ESI-SkoolBay · Junho de 2026

AUTORES:

António Rafael Marques Simões — nº2022115742

Diogo Filipe Gonçalves Pereira — nº2024115723

Kauã Henrique Santos Pina — nº2024126856

Leonardo Alexandre Martins Afonso — nº2022111829

Marinela Suely João Bettencourt — nº2024117541

Miguel Lourenço e Lozano Alves de Almeida Costa — nº2024107218

Conteúdo

Conteúdo.....	2
1. Descrição do Sistema Desenvolvido.....	3
1.1 Funcionalidades implementadas.....	3
1.2 Arquitetura e stack tecnológica	3
2. Tarefas do Processo Utilizadas e Justificação.....	4
3. Documentos de Requisitos.....	5
4. Resumo de Assiduidade	6
Diogo Pereira — Scrum Master / Dev Lead (10 issues).....	6
António Simões — QA / Backend de Serviços (11 issues).....	6
Kauã Pina — Modelação UML / Pedidos (9 issues).....	7
Leonardo Afonso — Autenticação / Testes / Documentação (10 issues).....	7
Marinela Bettencourt — UX / Frontend / Testes (10 issues).....	7
Miguel Costa — Infraestrutura / Backlog / Pedidos (10 issues).....	7
Trabalho coletivo de equipa (4 issues).....	8
Síntese.....	8

1. Descrição do Sistema Desenvolvido

O SkoolBay é uma plataforma web do tipo marketplace peer-to-peer que permite a estudantes universitários publicar, pesquisar e contratar serviços entre si — explicações, design, tradução, programação, revisão de textos, entre outros. A plataforma diferencia-se de marketplaces genéricos (Fiverr, OLX) por ser exclusivamente académica: o registo é restrito a endereços de e-mail institucionais verificados, e o sistema de reputação é construído dentro da própria comunidade universitária.

1.1 Funcionalidades implementadas

Foram implementadas as 20 User Stories planeadas (US-001 a US-020), distribuídas por cinco grandes áreas funcionais:

- **Autenticação e perfis** — registo com validação de domínio institucional, verificação de e-mail, login/logout com NextAuth.js v5 e gestão de perfil público com avatar e rating médio.
- **Gestão de serviços** — publicação, edição e remoção (soft delete) de serviços, listagem com paginação, pesquisa full-text, filtros por categoria e preço, e página de detalhe com avaliações.
- **Pedidos de serviço** — máquina de estados completa (PENDING → ACCEPTED/REJECTED → COMPLETED/CANCELLED), painel do prestador para gerir pedidos recebidos e painel do cliente para acompanhar pedidos enviados.
- **Reputação e moderação** — avaliações 1–5 estrelas após conclusão do serviço, recálculo automático do rating do prestador, denúncias de serviços/utilizadores e painel de moderação com gestão de domínios institucionais e de moderadores.
- **Funcionalidades com LLM** — pesquisa semântica inteligente na listagem de serviços e sugestão automática de preço no formulário de criação, através da API Groq (modelo llama-3.3-70b-versatile).

1.2 Arquitetura e stack tecnológica

Camada	Tecnologia
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind CSS + shadcn/ui
Backend	Next.js Route Handlers + Server Actions
Base de dados	PostgreSQL 16 + Prisma ORM
Autenticação	NextAuth.js v5 + bcrypt (salt 12)
Validação	Zod (schemas partilhados entre cliente e servidor)
IA / LLM	Groq API — llama-3.3-70b-versatile
Infraestrutura	Docker + Docker Compose (PostgreSQL local)

Camada	Tecnologia
CI/CD	GitHub Actions (lint, typecheck, testes)
Testes	Vitest (unitários) + testes de aceitação documentados por US

O modelo de dados assenta em 7 entidades Prisma (User, Category, Service, ServiceRequest, Review, Report e enum Role), com relações e máquina de estados documentadas nos diagramas UML. O controlo de versões seguiu uma política de branch protection na main, com todo o trabalho a passar por feature branches e Pull Requests com revisão obrigatória por um colega.

2. Tarefas do Processo Utilizadas e Justificação

Foram utilizadas as 17 tarefas do processo descrito no material “Instruções para o Projeto ESI”, mapeadas a issues no GitHub e organizadas em milestones alinhadas com as etapas do cronograma do professor (Etapas 0–7). A tabela seguinte resume cada tarefa e o motivo da sua utilização:

Tarefa	Descrição	Porquê foi utilizada
1–3	Criar repositório, GitHub Project e adicionar participantes	Base de todo o processo: sem repositório partilhado e quadro Kanban não há rastreabilidade nem colaboração. O GitHub Projects (To Do / In Progress / Done) foi o quadro Scrum da equipa.
4	Criar README.md	Ponto de entrada do projeto: descrição, participantes, stack e instruções de setup, essencial para qualquer pessoa executar o sistema.
5–7	Documento de Visão (docvisao.md)	Alinhou a equipa quanto a objetivo, escopo, stakeholders e arquitetura antes de escrever código, evitando ambiguidade durante a implementação.
8	Criar User Stories	As 20 US com critérios de aceitação e priorização MoSCoW foram a unidade de planeamento de todas as sprints e a base dos testes de aceitação.
9 e 11	Criar e refinar protótipos UX	Os 9 mockups (landing, registo, perfil, serviços, dashboard, avaliação) validaram o fluxo de utilização antes da implementação, reduzindo retrabalho de UI.
10	Refinar User Stories	O refinamento ajustou critérios de aceitação e story points com base na aprendizagem das primeiras sprints.
12	Diagrama de Casos de Uso	Modelou 24 casos de uso e 5 atores (visitante, cliente, prestador, moderador, admin), clarificando permissões e

Tarefa	Descrição	Porquê foi utilizada
		fronteiras do sistema.
13	Diagrama de Classes	Definiu as 7 entidades, relações e a máquina de estados do pedido — serviu de especificação direta para o schema Prisma.
14	Implementação (Sprints 1–4)	Concretização das 20 US em código funcional, com PRs por User Story e CI a validar cada merge.
15	Testes Unitários	37 testes Vitest sobre validações Zod e lógica de autenticação/serviços/pedidos, todos a passar no CI.
16	User Stories como Testes de Aceitação	Cada US foi executada como teste manual documentado (passos, resultado esperado, resultado real), validando o produto contra os requisitos.
17	Retrospectiva e Relatório Final	Consolidação das lições aprendidas, métricas e contribuições — entregue como documento autónomo em anexo.

Todas as tarefas foram registadas como issues no GitHub (64 issues no total), associadas a milestones por sprint e fechadas através de Pull Requests com a referência “Closes #N”, garantindo rastreabilidade completa entre o processo definido pelo professor e o trabalho efetivamente realizado.

3. Documentos de Requisitos

Toda a documentação de requisitos encontra-se versionada no repositório, na pasta docs/:

Documento	Conteúdo e localização
Documento de Visão	docs/docvisao.md — 8 secções: objetivo, escopo, stakeholders, equipa, funcionalidades, arquitetura, restrições e integração LLM.
Product Backlog / User Stories	docs/scrum/product-backlog.md — US-001 a US-020 com critérios de aceitação, prioridades MoSCoW, story points (86 no total) e atribuição a sprints.
Diagrama de Casos de Uso	docs/uml/casos-de-uso.md — diagrama Mermaid com 24 casos de uso e 5 atores, com descrição de cada UC.
Diagrama de Classes	docs/uml/diagrama-classes.md — 7 entidades, enums, relações e máquina de estados do ServiceRequest.

Documento	Conteúdo e localização
Protótipos UX	docs/ux/ — 9 mockups (landing, registo, login, perfil, criação de serviço, listagem, detalhe, dashboard, avaliação).
Testes de Aceitação	docs/test/acceptance-tests.md — execução documentada das User Stories como testes de aceitação.
Retrospectiva	docs/scrum/retrospectiva.md — retrospectiva final do projeto (ver também o documento anexo da Tarefa 17).

4. Resumo de Assiduidade

A distribuição do trabalho seguiu os papéis definidos no início do projeto, com todas as tarefas registadas e atribuídas no GitHub Projects. Nota: o grupo iniciou com 7 elementos; um elemento saiu numa fase inicial, antes da atribuição individual de issues, constando apenas das issues coletivas de arranque, sem impacto no âmbito entregue pelos 6 membros ativos. A lista seguinte indica, por integrante, as issues efetivamente atribuídas e fechadas no GitHub. No total foram 64 issues: 60 atribuídas individualmente e 4 issues de arranque partilhadas por toda a equipa.

Diogo Pereira — Scrum Master / Dev Lead (10 issues)

- #3 — Documento de Visão (docvisao.md)
- #4 — Setup Next.js + TypeScript + Tailwind + shadcn/ui
- #5 — Configurar Prisma ORM + PostgreSQL
- #14 — Product Backlog e Definition of Done
- #19 — Layout base e navegação
- #20 — Diagrama de Sequência — Fluxo de Registo
- #32 — Refinar Protótipos UX
- #34 — Sistema de avaliações
- #39 — Testes Unitários — auth e user
- #60 — US-014 — Avaliar prestador

António Simões — QA / Backend de Serviços (11 issues)

- #11 — Diagrama de Classes
- #13 — Diagrama de Estados — ServiceRequest
- #17 — Página de Login
- #21 — CRUD de serviços — criação
- #23 — Página de detalhe de serviço
- #27 — Enviar pedido de serviço
- #41 — User Stories como Testes de Aceitação
- #51 — US-005 — Publicar serviço
- #52 — US-006 — Editar serviço
- #53 — US-007 — Remover serviço
- #65 — US-019 — Navegação e layout base

Kauã Pina — Modelação UML / Pedidos (9 issues)

- #10 — Diagrama de Casos de Uso
- #28 — Painel do prestador — gestão de pedidos
- #29 — Painel do cliente — pedidos enviados
- #33 — Diagrama de Sequência — Contratação e Avaliação
- #35 — Componente de estrelas (rating)
- #54 — US-008 — Pesquisar e filtrar serviços
- #62 — US-016 — Denunciar serviço ou utilizador
- #64 — US-018 — Dashboard principal
- #66 — US-020 — Landing page

Leonardo Afonso — Autenticação / Testes / Documentação (10 issues)

- #16 — Registo com validação de e-mail institucional
- #24 — Listagem de serviços com pesquisa e filtros
- #42 — Documento de Retrospectiva e Relatório Final
- #43 — Documentação final completa
- #44 — Preparação da Apresentação Final
- #47 — US-001 — Registo com e-mail institucional
- #48 — US-002 — Login na plataforma
- #49 — US-003 — Logout
- #50 — US-004 — Ver e editar perfil
- #55 — US-009 — Ver detalhe de serviço

Marinela Bettencourt — UX / Frontend / Testes (10 issues)

- #9 — Protótipos de UX (Mockups)
- #18 — Perfil de utilizador (view + edit)
- #25 — Gestão de categorias (seed + UI)
- #26 — Diagrama de Sequência — Publicação de Serviço
- #30 — Dashboard principal
- #31 — Refinar User Stories
- #37 — Painel de moderação (admin)
- #38 — Polish de UI e responsividade
- #40 — Testes Unitários — service e request
- #56 — US-010 — Enviar pedido de serviço

Miguel Costa — Infraestrutura / Backlog / Pedidos (10 issues)

- #7 — Configurar GitHub Actions (CI/CD)
- #12 — Diagrama de Componentes
- #15 — NextAuth.js v5 com credentials provider
- #22 — CRUD de serviços — edição e remoção
- #36 — Sistema de denúncia
- #57 — US-011 — Gerir pedidos recebidos
- #58 — US-012 — Ver pedidos enviados
- #59 — US-013 — Cancelar pedido
- #61 — US-015 — Ver avaliações de prestador

- #63 — US-017 — Moderar conteúdo (admin)

Trabalho coletivo de equipa (4 issues)

As issues de arranque do projeto foram atribuídas a toda a equipa: #1 Criar repositório GitHub, #2 Criar README.md, #6 Configurar Docker + Docker Compose e #8 Criar User Stories completas. O elemento que abandonou o projeto (Jonathan) consta apenas destas issues coletivas iniciais.

Síntese

Integrante	Função principal	Issues
Diogo Pereira	Scrum Master / Dev Lead	10
António Simões	QA / Backend de Serviços	11
Kauã Pina	Modelação UML / Pedidos	9
Leonardo Afonso	Autenticação / Testes / Documentação	10
Marinela Bettencourt	UX / Frontend / Testes	10
Miguel Costa	Infraestrutura / Backlog / Pedidos	10